

1. Object-Relational Mapping (ORM)

Definition: A programming technique for converting data between incompatible type systems in object-oriented programming languages and relational databases.

Benefits:

- Abstraction: Hides SQL complexity, allowing developers to interact purely with Java objects.
- Productivity: Eliminates tedious, boilerplate JDBC code (opening connections, handling statements, and manually parsing results).
- Database Independence: Translates application-level queries into database-specific SQL dialect automatically.
- Transaction Management: Integrates seamlessly with advanced caching, dirty checking, and transaction contexts.
- 2. Core Framework Specifications & Implementations
- JPA (Java Persistence API): The formal, standard specification (JSR 338) defining how relational data should be managed in Java applications. It contains interfaces but zero implementation code.
- Hibernate: A complete ORM framework that serves as a concrete implementation of the JPA specification.
- Spring Data JPA: A high-level abstraction layer built over Hibernate. It removes the necessity of writing data-access-layer boilerplate code by automatically generating CRUD repository implementations at runtime.

3. Core Native Objects of Hibernate

- SessionFactory: A thread-safe, application-wide cache of compiled mappings used to generate Session instances.
- Session: A short-lived, single-threaded object representing a conversational unit of work between the application and the persistent store.
- Transaction: A single-threaded, short-lived object used by the application to specify atomic units of work.
- ConnectionProvider: A factory for abstracting JDBC connections from standard underlying datasources or drivers.

Database Initialization

```
CREATE SCHEMA ormlearn;
```

```
USE ormlearn;
```

```
CREATE TABLE country (  
    co_code VARCHAR(2) PRIMARY KEY,  
    co_name VARCHAR(50)  
);
```

```
INSERT INTO country VALUES ('IN', 'India');
```

```
INSERT INTO country VALUES ('US', 'United States of America');
```

Configuration File (application.properties)

```
logging.level.org.springframework=info
```

```
logging.level.com.cognizant=debug
```

```
logging.level.org.hibernate.SQL=trace
```

```
logging.level.org.hibernate.type.descriptor.sql=trace
```

```
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-  
20.20thread %5p %-25.25logger{25} %25M %4L %m%n
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
```

```
spring.datasource.username=root
```

```
spring.datasource.password=root
```

```
spring.jpa.hibernate.ddl-auto=validate
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5  
Dialect
```

Persistence Model Class (Country.java)

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="country")
public class Country {

    @Id
    @Column(name="co_code")
    private String code;

    @Column(name="co_name")
    private String name;

    public Country() {}

    public String getCode() { return code; }

    public void setCode(String code) { this.code = code; }
```

```
public String getName() { return name; }

public void setName(String name) { this.name = name; }

@Override

public String toString() {

    return "Country [code=" + code + ", name=" + name + "];"

}

}
```

Data Repository Component (CountryRepository.java)

```
package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository

public interface CountryRepository extends JpaRepository<Country,
String> {

}
```

Application Service Layer (CountryService.java)

```
package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional(readOnly = true)
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}
```